

Name : Kanika Saini

SAP ID : 590034098

Batch : B16

Program : Btech CSE

Experiment-3

C Programming - Debugging Tools

1. Fundamentals of Errors and Debugging

Key Definitions-

Error: An error is a mistake in a program that causes it to produce an incorrect result, behave unexpectedly, or fail to compile or execute.

Example: Missing a semicolon (;) in a C program.

Bug: A bug is a defect or fault in the program's code that causes the program to behave differently from what was intended.

Example: Using + instead of * while calculating the area of a rectangle.

Debugging: Debugging is the process of finding, analyzing, and correcting errors or bugs in a program. Debugging tools help programmers locate problems more easily.

In simple words:

Error = the mistake → Bug = the defect caused/present in the code → Debugging = finding and fixing the problem.

2. Classification of Errors

1. Syntax Error

Description: An error caused by violating the grammar rules of the C language. The compiler detects these errors before execution.

Primary Cause: Missing semicolons, unmatched brackets, misspelled keywords, or incorrect statement syntax.

Example: `printf("Hello")` (missing `;`)

2. Compilation / Type Error

Description: An error where the program has correct syntax but the meaning of the code is incorrect.

Primary Cause: Incorrect use of variables, data types, operators, or functions.

Example: `int x = "10";` (assigning a string to an integer variable)

3. Runtime Error

Description: An error that occurs while the program is running, causing abnormal behavior or program termination.

Primary Cause: Division by zero, invalid memory access, null pointers, or missing files.

Example: `int a = 10, b = 0; printf("%d", a / b);`

4. Logical Error

Description: An error in the program's logic that produces incorrect output even though the program compiles and runs successfully.

Primary Cause: Wrong algorithm, incorrect formula, or incorrect conditions.

Example: Calculating the area of a circle as $2 * \pi * r$ instead of $\pi * r^2$.

Why Does a Program Compile Successfully but Produce Incorrect Output?

A program may **compile successfully but produce incorrect output** because it contains a **logical error**. In this case, the program follows the correct C syntax, so the compiler finds no errors. However, the logic or algorithm used in the program is incorrect, causing it to generate wrong results during execution.

Example: If the area of a circle is calculated as $2 \times \pi \times r$ instead of $\pi \times r^2$, the program compiles and runs without errors, but the output is incorrect because the formula is wrong.

Case 1: Syntax & Compilation Errors

When the compiler encounters invalid code, it prints diagnostics specifying the file name, line number, column, and error description.

Example Code (sample1.c)

```
#include <stdio.h>

int main()

{

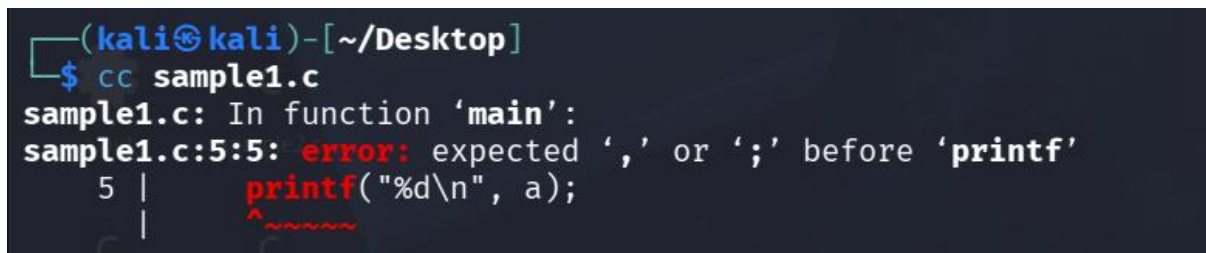
    int a = 10

    printf("%d\n", a);

    return 0;

}
```

Compilation Command & Output

A terminal window with a dark background. The prompt is (kali@kali)-[~/Desktop]. The command \$ cc sample1.c has been entered. The output shows an error: sample1.c: In function 'main': sample1.c:5:5: error: expected ',', or ';' before 'printf'. Below this, line 5 is shown with a vertical line at column 5 pointing to the start of the printf statement. A red squiggly line is under the printf function name.

```
(kali@kali)-[~/Desktop]
$ cc sample1.c
sample1.c: In function 'main':
sample1.c:5:5: error: expected ',', or ';' before 'printf'
  5 |     printf("%d\n", a);
    |     ^~~~~~
```

Breakdown error message

- sample1.c → Name of the C source file containing the error.
- 5:5 → The error is detected at line 5, column 5.
- error: → Indicates that the compiler found a problem that prevents successful compilation.
- expected ',', or ';' → The compiler expected a comma or semicolon before this point.
- before 'printf' → The compiler noticed the problem when it reached the printf() statement.
- '~~~~~' under printf → Shows the location where the compiler detected the error.

Correct code -

```
#include <stdio.h>

int main()
{
    int a = 10;                // error resolved
    printf("%d\n", a);
    return 0;
}
```

Case 2: Compiler Warnings (-Wall, -Wextra)

A warning indicates code that is syntactically legal, but potentially dangerous or suspicious. Warnings do not stop compilation, but ignoring them frequently leads to bugs at runtime.

Example Code (sample2.c)

```
#include <stdio.h>

int main()
{
    int a;
    printf("Hello\n");
    return 0;
}
```

Standard Compilation vs. Warning Flags-

```

(kali@kali)-[~/Desktop]
$ cc sample2.c

(kali@kali)-[~/Desktop]
$ ./a.out
Hello

(kali@kali)-[~/Desktop]
$ cc -Wall -Wextra sample2.c
sample2.c: In function 'main':
sample2.c:4:9: warning: unused variable 'a' [-Wunused-variable]
    4 |     int a;
      |         ^

```

Breakdown of the Warning Message

- sample2.c → File in which the problem is found.
- 4:9 → Warning is detected at line 4, column 9.
- warning: → It is a warning, not a fatal error. The program can still compile.
- unused variable 'a' → Variable a was declared but is never used.
- [-Wunused-variable] → The specific compiler warning responsible for detecting an unused variable.
- ^ symbol → Points to the exact variable (a) causing the warning.

-Wall: Enables a broad set of commonly useful compiler warnings.

-Wextra: Enables additional warnings not covered by -Wall.

Correct code –

```
#include <stdio.h>
```

```
int main()
```

```
{  
    printf("Hello\n");  
    return 0;  
}
```

Compile again:

```
$ cc -Wall -Wextra sample2.c
```

Now, no warning should appear.

Case 3: printf-Based Debugging (Logical bug)

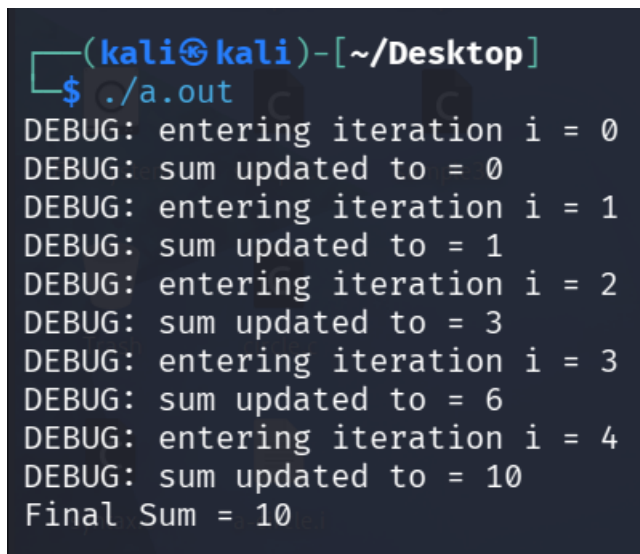
Example: Tracking Loop Execution (sample3.c)

The following program adds numbers from 1 to 4 instead of 1 to 5 due to a logical bug. Debug printf() statements help expose the exact iteration behavior.

```
#include <stdio.h>  
  
int main(void)  
{  
    int i, sum = 0;  
    for(i = 0; i < 5; i++)  
    {  
        printf("DEBUG: entering iteration i = %d\n", i);  
        sum = sum + i;  
        printf("DEBUG: sum updated to = %d\n", sum);  
    }  
}
```

```
}  
  
printf("Final Sum = %d\n", sum);  
  
return 0;  
  
}
```

Output after compilation and execution-



A terminal window with a dark background. The prompt is `(kali㉿kali)-[~/Desktop]`. The user enters `./a.out`. The program outputs several debug messages: `DEBUG: entering iteration i = 0`, `DEBUG: sum updated to = 0`, `DEBUG: entering iteration i = 1`, `DEBUG: sum updated to = 1`, `DEBUG: entering iteration i = 2`, `DEBUG: sum updated to = 3`, `DEBUG: entering iteration i = 3`, `DEBUG: sum updated to = 6`, `DEBUG: entering iteration i = 4`, `DEBUG: sum updated to = 10`, and finally `Final Sum = 10`.

```
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
DEBUG: entering iteration i = 0  
DEBUG: sum updated to = 0  
DEBUG: entering iteration i = 1  
DEBUG: sum updated to = 1  
DEBUG: entering iteration i = 2  
DEBUG: sum updated to = 3  
DEBUG: entering iteration i = 3  
DEBUG: sum updated to = 6  
DEBUG: entering iteration i = 4  
DEBUG: sum updated to = 10  
Final Sum = 10
```

What is the Error?

The error is in the loop condition:

```
for(i = 0; i < 5; i++)
```

The loop takes these values:

$i = 0, 1, 2, 3, 4$

When i becomes 5, the condition $i < 5$ becomes false. Therefore, 5 is never added to the sum.

So the program calculates:

$$0 + 1 + 2 + 3 + 4 = 10$$

instead of:

$1 + 2 + 3 + 4 + 5 = 15$

This is an off-by-one logical error.

Why Doesn't the Compiler Show an Error?

The compiler does not report an error because:

```
for(i = 0; i < 5; i++)
```

is perfectly valid C syntax.

The compiler does not know that our intention was to calculate the sum from 1 to 5. Therefore, the program compiles and runs normally but gives the wrong answer.

That's why this is a logical error, not a syntax or compilation error.

Correct Code

For the intended sum from 1 to 5, a clearer correction is:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int i, sum = 0;
```

```
    for(i = 1; i <= 5; i++)
```

//error resolved

```
    {
```

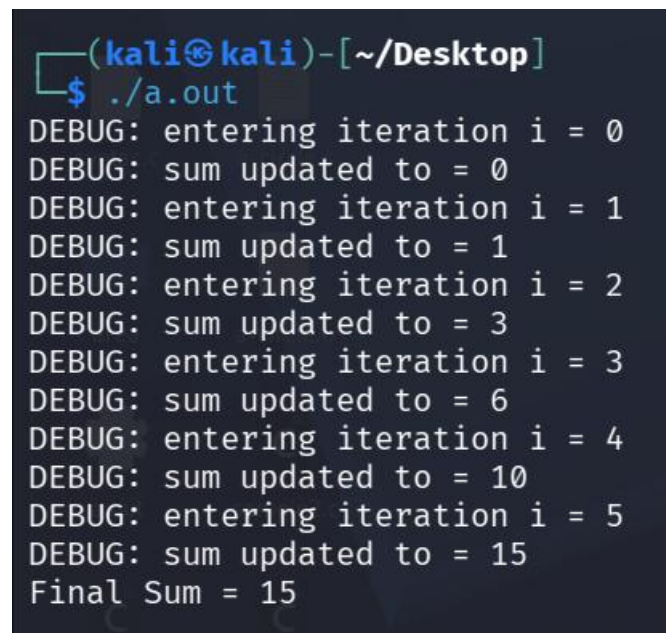
```
        printf("DEBUG: entering iteration i = %d\n", i);
```

```
        sum = sum + i;
```

```
        printf("DEBUG: sum updated to = %d\n", sum);
```

```
}  
  
printf("Final Sum = %d\n", sum);  
  
return 0;  
  
}
```

Correct output-



A terminal window screenshot from a Kali Linux system. The prompt is `(kali㉿kali)-[~/Desktop]`. The user has executed `./a.out`. The program outputs a series of debug messages showing iterations from 0 to 5, with the sum being updated at each step. The final output is `Final Sum = 15`.

```
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
DEBUG: entering iteration i = 0  
DEBUG: sum updated to = 0  
DEBUG: entering iteration i = 1  
DEBUG: sum updated to = 1  
DEBUG: entering iteration i = 2  
DEBUG: sum updated to = 3  
DEBUG: entering iteration i = 3  
DEBUG: sum updated to = 6  
DEBUG: entering iteration i = 4  
DEBUG: sum updated to = 10  
DEBUG: entering iteration i = 5  
DEBUG: sum updated to = 15  
Final Sum = 15
```

3. Introduction to GDB (GNU Debugger)

What is GNU?

GNU stands for “GNU’s Not Unix.” It is a free and open-source software project that provides many programming tools and utilities. GCC and GDB are important tools associated with the GNU Project.

What is GDB?

GDB stands for GNU Debugger. It is a command-line debugging tool used to find and analyze errors in programs, especially programs written in languages such as C and C++.

With GDB, a programmer can:

- Run a program under the debugger.
- Set breakpoints to pause execution at specific lines.
- Execute the program line by line.
- Check the current values of variables.
- Observe how variable values change during execution.
- Find where a program crashes or behaves incorrectly.

Installation & Compilation for GDB

Update package list and install GDB on Linux systems

```
$ sudo apt update
```

```
$ sudo apt install gdb
```

Compile source code with debugging symbols enabled (-g)

```
$ cc -g -Wall -Wextra program.c -o program
```

Launch GDB with the compiled binary

```
$ gdb ./program
```

Quick Reference: Common GDB Commands

1. break <line/function>

Shorthand: b

Purpose: Sets a breakpoint at a specified line number or function name.

2. run

Shorthand: r

Purpose: Starts execution of the program from the beginning.

3. next

Shorthand: n

Purpose: Executes the next line of code and steps over function calls.

4. step

Shorthand: s

Purpose: Executes the next line of code and steps into function calls.

5. print <var>

Shorthand: p

Purpose: Prints the current value of a variable or expression.

6. display <var>

Shorthand: disp

Purpose: Automatically prints the value of a variable after every step.

7. continue

Shorthand: c

Purpose: Resumes program execution until the next breakpoint or until the program finishes.

8. list

Shorthand: l

Purpose: Displays source-code lines around the current execution position.

9. info locals

Shorthand: info loc

Purpose: Displays all local variables and their current values in the current stack frame.

10. backtrace

Shorthand: bt

Purpose: Displays the current call stack, showing the chain of function calls.

11. info breakpoints

Shorthand: i b

Purpose: Displays all active breakpoints and their numbers.

12. delete <id>

Shorthand: d

Purpose: Deletes the specified breakpoint. If the ID is omitted, GDB can delete all breakpoints after confirmation.

13. disable <id>

Shorthand: dis

Purpose: Temporarily disables a breakpoint without deleting it.

14. enable <id>

Shorthand: en

Purpose: Re-enables a previously disabled breakpoint.

15. quit

Shorthand: q

Purpose: Exits the GDB debugging session.

Case 4: Debugging a Logical Error using GDB

Example: Code with Logic Bug (sample4.c)

```
#include <stdio.h>                                // Line 1
int main()                                          // Line 2
{                                                  // Line 3
    int i, sum = 0;                                // Line 4
    for(i = 1; i < 5; i++)                          // Line 5 (Logical Bug: should
be i <= 5)
    {                                              // Line 6
        sum = sum + i;                             // Line 7
```

```
} // Line 8  
printf("Sum = %d\n", sum); // Line 9  
return 0; // Line 10  
}
```



```
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
Sum = 10
```

- **Expected Output:** 15 (1+2+3+4+5)
- **Actual Output:** 10 (1+2+3+4)

Debugging walkthrough in GDB

We used GDB to find the logical error in the program by performing the following steps:

1. The program was compiled using `cc -g sample4.c` to include debugging information.
2. GDB was started using `gdb ./a.out`.
3. A breakpoint was set at line 7 (`sum = sum + i;`) using `break 7`.
4. The program was started with the `run` command.
5. `print i` was used to check the current value of `i`.
6. `display i` and `display sum` were used to automatically observe the values of `i` and `sum`.
7. Using `continue`, we observed the loop repeatedly. GDB showed `i = 1, 2, 3, 4`, but never executed the addition for `i = 5`.

Bug Found: The condition $i < 5$ stops the loop before 5 is added. Therefore, the logical bug was identified and corrected to $i \leq 5$, changing the final sum from 10 to 15.

```
(gdb) list
1      #include <stdio.h>                // Line 1
2      int main()                        // Line 2
3      {                                // Line 3
4          int i, sum = 0;                // Line 4
5          for(i = 1; i < 5; i++)         // Line 5 (Logical Bug: should be i ≤ 5)
6          {                                // Line 6
7              sum = sum + i;             // Line 7
8          }                                // Line 8
9          printf("Sum = %d\n", sum);     // Line 9
10         return 0;                      // Line 10
(gdb) break 7
Breakpoint 1 at 0x1151: file sample4.c, line 7.
(gdb) run
Starting program: /home/kali/Desktop/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;                // Line 7
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;                // Line 7
1: i = 2
2: sum = 1
(gdb) continue
```



```

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;           // Line 7
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;           // Line 7
1: i = 2
2: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;           // Line 7
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;           // Line 7
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
Sum = 10
[Inferior 1 (process 4882) exited normally]
(gdb) █

```

Correct Code

```

#include <stdio.h>

int main()
{
    int i, sum = 0;

```

```
for(i = 1; i <= 5; i++)  
{  
    sum = sum + i;  
}  
printf("Sum = %d\n", sum);  
return 0;  
}
```

//error resolved

The correction is:

Incorrect: $i < 5$

Correct: $i \leq 5$

Now the loop executes for:

1, 2, 3, 4, 5

Therefore:

$1 + 2 + 3 + 4 + 5 = 15$

Correct Output



A terminal window with a dark background. The prompt is `(kali@kali)-[~/Desktop]`. The user has entered `$ ./a.out`. The output of the program is `Sum = 15`.

Case 5: Debugging Functions (next vs. step)

When debugging a C program using GDB (GNU Debugger), the commands next and step are used to execute the program one line at a time. The main difference is how they handle function calls.

1. next (or n)

Definition:

The next command executes the current line of code and moves to the next line. If the current line contains a function call, GDB executes the entire function and stops at the next line in the current function.

Use:

Use next when you do not need to inspect what happens inside the called function.

Example:

```
sum = add(a, b);  
printf("%d", sum);
```

If you type:

```
(gdb) next
```

GDB executes the complete add() function and then stops at:

```
printf("%d", sum);
```

2. step (or s)

Definition:

The step command executes the current line one step at a time. If the current line contains a function call, GDB enters the function and allows you to debug its statements line by line.

Use:

Use step when you want to examine how a function works internally or find bugs inside it.

Example:

```
sum = add(a, b);
```

If you type:

```
(gdb) step
```

GDB enters the add() function and stops at its first executable line.

Difference Between next and step

- next executes the current line and does not enter called functions.
- step executes the current line and enters called functions.
- next is useful when the called function is already known to work correctly.
- step is useful when you suspect the bug is inside the function and want to inspect it line by line.

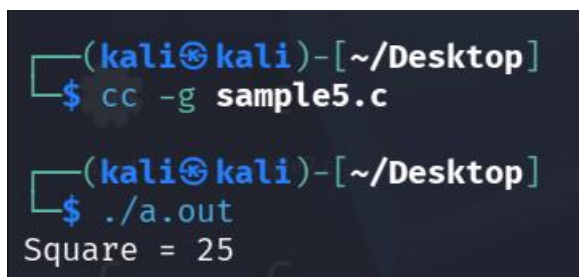
Example Code (sample5.c)

```
#include <stdio.h>
```

```
int square(int x)
{
    int result1;
    result1 = x * x;
    return result1;
}

int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
    return 0;
}
```

Compilation and output-

A terminal window with a dark background and light blue text. The prompt is (kali@kali)-[~/Desktop]. The first command is \$ cc -g sample5.c. The second command is \$./a.out, followed by the output Square = 25.

```
(kali@kali)-[~/Desktop]
$ cc -g sample5.c

(kali@kali)-[~/Desktop]
$ ./a.out
Square = 25
```

Our program is correct. There is no syntax error and it will compile and run successfully. The purpose of this case is not to fix a broken program, but to learn how to debug a C program using GDB.

Option A: Stepping over with next

(gdb) **break main** → Sets a breakpoint at the main() function.

(gdb) **run** → Starts the program and stops at the breakpoint.

(gdb) **next** → Executes line 10 and moves to the next line.

(gdb) **next** → Executes square(n) without entering the square() function.

(gdb) **next** → Executes the printf() statement and prints Square = 25.

(gdb) **print result** → Displays the value of result (25)

Using next, GDB executes each line without entering function calls, making it useful for quickly debugging the main program.

```
Reading symbols from ./a.out ...
(gdb) break main
Breakpoint 1 at 0x1156: file sample5.c, line 10.
(gdb) run
Starting program: /home/kali/Desktop/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample5.c:10
10      int result, n = 5;
(gdb) next
11      result = square(n);
(gdb) next
12      printf("Square = %d\n", result);
(gdb) next
Square = 25
13      return 0;
(gdb) print result
$1 = 25
(gdb)
```

B : Stepping into with step

(gdb) **break main** → Sets a breakpoint at the main() function.

(gdb) run → Starts the program and stops at the breakpoint.

(gdb) next → Executes the current line without entering the function.

(gdb) step → Enters the square() function to debug it line by line.

(gdb) square(x=5) at sample5.c:5 → GDB enters square() with x = 5.

(gdb) print x → Displays the current value of x (5).

(gdb) next → Executes the multiplication and moves to the return statement.

(gdb) print result1 → Displays the value of result1 (25)

```
Reading symbols from ./a.out...
(gdb) break main
Breakpoint 1 at 0x1156: file sample5.c, line 10.
(gdb) run
Starting program: /home/kali/Desktop/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample5.c:10
10      int result, n = 5;
(gdb) next
11      result = square(n);
(gdb) step
square (x=5) at sample5.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6      return result1;
(gdb) print result1
$2 = 25
(gdb)
```

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function call sequence leading to the crash.

Example Code (sample6.c)

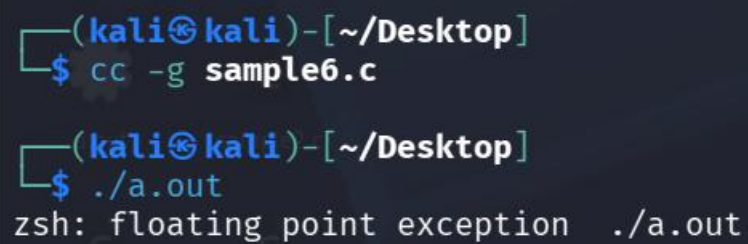
```
#include <stdio.h>

int divide(int a, int b)
{
    return a / b;                // Division by zero when b = 0
}

int calculate(int x)
{
    return divide(x, 0);        // Passes 0 as divisor
}

int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

Compilation and output (error)



```
(kali㉿kali)-[~/Desktop]
$ cc -g sample6.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
zsh: floating point exception ./a.out
```


This program contains a runtime error.

Objective: This program divides a number and prints the result.

Bug in the Code: The bug is here-

```
' return divide(x, 0); '
```

The function passes 0 as the divisor, so divide() executes:

```
return a / b;
```

Since $b = 0$, the program performs division by zero, which causes a runtime error (Floating Point Exception).

GDB Backtrace Execution

```
$ gdb ./a.out
```

(gdb) run → Starts the program in debugging mode.

(gdb) backtrace → Shows the sequence of function calls that caused the error.

GDB Output-

- The program stopped due to an arithmetic error (division by zero).
- `divide(a=10, b=0)` → Error occurred in the `divide()` function because $b = 0$.

Understand stack frames -

- Frame #0 – `divide()` → The runtime error occurred here due to division by zero.
- Frame #1 – `calculate()` → This function called `divide(10, 0)`.
- Frame #2 – `main()` → This is the starting function that called `calculate()`.

Call sequence: main() → calculate() → divide() (where the error occurred).

```
Reading symbols from ./a.out ...
(gdb) run
Starting program: /home/kali/Desktop/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x0000555555555147 in divide (a=10, b=0) at sample6.c:4
4      return a / b;           // Division by zero when b = 0
(gdb) backtrace
#0  0x0000555555555147 in divide (a=10, b=0) at sample6.c:4
#1  0x0000555555555166 in calculate (x=10) at sample6.c:8
#2  0x000055555555517a in main () at sample6.c:12
```

Correct Code

We have to use a non-zero divisor to correct the code

```
#include <stdio.h>
```

```
int divide(int a, int b)
```

```
{
```

```
    return a / b;
```

```
}
```

```
int calculate(int x)
```

```
{
```

```
    return divide(x, 2);           // error resolved
```

```
}
```

```
int main()
```

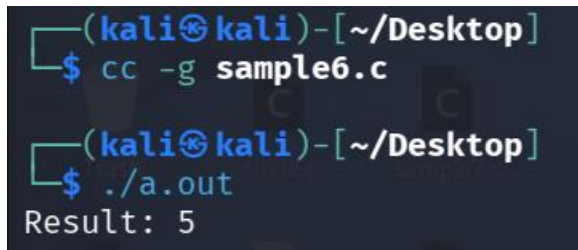
```
{
```

```
    int result = calculate(10);
```

```
    printf("Result: %d\n", result);
```

```
    return 0;
}
```

Now the output is-

A terminal window with a dark background. The prompt is (kali㉿kali)-[~/Desktop]. The first command is \$ cc -g sample6.c. The second command is \$./a.out. The output is Result: 5.

```
(kali㉿kali)-[~/Desktop]
$ cc -g sample6.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
Result: 5
```

Case 7: Debugging Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked at runtime automatically. Accessing indices beyond the declared limit results in undefined behavior or memory corruption.

Example Code (sample7.c)

```
#include <stdio.h>

int main()
{
    int a[5];

    int i;

    for(i = 0; i <= 5; i++)
    {
        scanf("%d", &a[i]); //Line 8
    }
}
```

```
    return 0;
}
```

This program contains a bug (Array Out of Bounds Error).

The given code has no printed output. It only takes input, and due to the bug, its behavior after the fifth input is undefined.

Objective- This program is used to take input for an array of 5 integers.

Bug in the Code- The bug is in the for loop:

```
for(i = 0; i <= 5; i++)
```

The condition `i <= 5` is incorrect. The array is declared as: `int a[5];`

It has valid indexes 0, 1, 2, 3, 4 only. When `i` becomes 5, the program tries to access: `a[5]` which is outside the array boundary.

This is called an Array Out of Bounds Error.

Inspection in GDB

(gdb) break 8 → Sets a breakpoint at line 8.

(gdb) run → Starts the program and stops at line 8.

(gdb) print &a[0] → Displays the memory address of the first array element.

(gdb) print &a[4] → Displays the memory address of the last valid array element.

(gdb) print &a[5] → Displays the address of an out-of-bounds array element, indicating invalid access.

```
Reading symbols from ./a.out...
(gdb) break 8
Breakpoint 1 at 0x114a: file sample7.c, line 8.
(gdb) run
Starting program: /home/kali/Desktop/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample7.c:8
warning: Source file is more recent than executable.
8      scanf("%d", &a[i]); //Line 8
(gdb) print &a[0]
$1 = (int *) 0x7fffffffdd20
(gdb) print &a[4]
$2 = (int *) 0x7fffffffdd30
(gdb) print &a[5]
$3 = (int *) 0x7fffffffdd34
(gdb) █
```

Correct Code

Correct Loop Condition- for(i = 0; i < 5; i++)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a[5];
```

```
    int i;
```

```
    for(i = 0; i < 5; i++)
```

//error resolved

```
    {
```

```
        scanf("%d", &a[i]); //Line 8
```

```
    }
```

```
    return 0;
```

```
}
```

Now the program accepts exactly 5 numbers and exits normally.

Case 8 : Segmentation Faults (SIGSEGV) using GDB

A Segmentation Fault occurs when a program attempts to access a memory location that it does not have permission to access (such as dereferencing a NULL pointer).

Example Code (sample8.c)

```
#include <stdio.h>

int main()
{
    int *p = NULL;    // Pointer points to address 0x0

    *p = 10;          // Attempting to write to address 0x0 causes
Segmentation Fault

    printf("%d\n", *p);

    return 0;
}
```

Bug in the Code-

```
int *p = NULL;

*p = 10;
```

p points to NULL (address 0x0). Writing 10 to *p causes a Segmentation Fault (SIGSEGV) because address 0x0 is invalid.

Debugging in GDB

- (gdb) run → Starts the program in debugging mode.
- (gdb) print p → Displays the value of pointer p (0x0, which is NULL).
- (gdb) print * p → Tries to access the value pointed to by p and shows an error because p is NULL.

GDB Output

- Program received signal SIGSEGV → The program crashed due to a segmentation fault.
- 'Cannot access memory at address 0x0' → GDB confirms that p points to an invalid memory address (NULL).

```
Reading symbols from ./a.out...
(gdb) run
Starting program: /home/kali/Desktop/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at sample8.c:5
5          *p = 10;           // Attempting to write to address 0x0 causes Segmentation Fault
(gdb) print p
$p1 = (int *) 0x0
(gdb) print *p
✗ Cannot access memory at address 0x0
```

Correct code

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int x;
```

```
int *p = &x;        // Assign p to point to valid memory address of x

*p = 10;            // Valid write

printf("%d\n", *p);

return 0;

}
```

Output –

A terminal window with a dark background and light blue text. The prompt is '(kali㉿kali)-[~/Desktop]'. The first command is '\$ cc -g sample8.c'. The second command is '\$./a.out', followed by the output '10' on a new line.

```
(kali㉿kali)-[~/Desktop]
$ cc -g sample8.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
10
```

4. Systematic Debugging Strategy

To debug effectively, follow this 8-step methodology:

1. Observe Expected vs. Actual Output
2. Reproduce the Issue Consistently
3. Compile with Flags (-Wall -Wextra -g)
4. Isolate Suspicious Code Sections
5. Inspect State (printf or GDB breakpoints)
6. Apply a Single Minimal Fix
7. Recompile and Re-test
8. Verify with Edge Cases & Boundary Inputs

5. Exercises & Homework Questions

Q1. Error Classification: Write a small C program snippet illustrating each of the following error types: Syntax error, Compilation/Type error, Linker error, Runtime error, and Logical error.

ANS- Explained above.

Q2. GNU Concept: Explain what the GNU project is and how GDB fits into the GNU ecosystem.

ANS- Explained above.

Q3. Shell Context: Define what a Shell and Kernel are. Briefly list three common Linux shells (e.g., bash, zsh, fish).

ANS- A **Shell** is a command-line interface that allows users to communicate with the Linux operating system by entering commands.

The **Kernel** is the core of the operating system. It manages hardware resources, memory, processes, and communication between software and hardware.

Three Common Linux Shells

Bash (Bourne Again Shell) – The default and most widely used Linux shell.

Zsh (Z Shell) – An advanced shell with improved features like auto-completion and customization.

Fish (Friendly Interactive Shell) – A user-friendly shell with syntax highlighting and command suggestions.

Q4. Research and contrast the differences between: Breakpoint, Watchpoint, Catchpoint, and Tracepoint.

ANS- 1. Breakpoint

A Breakpoint is a marker that pauses program execution at a specific line of code or function, allowing you to inspect variables and program state.

2. Watchpoint

A Watchpoint stops the program whenever the value of a specified variable or memory location changes. It is useful for tracking unexpected data modifications.

3. Catchpoint

A Catchpoint pauses the program when a specific event occurs, such as an exception, signal, or system call, helping to debug runtime events.

4. Tracepoint

A Tracepoint records program information when execution reaches a certain point without stopping the program. It is useful for collecting debugging data while the program continues to run.

Difference -

- Breakpoint → Stops execution at a specific line or function.
 - Watchpoint → Stops when a variable or memory value changes.
 - Catchpoint → Stops when a specific event (signal, exception, etc.) occurs.
 - Tracepoint → Records execution information without stopping the program.
-